TECHNICAL TRAINING COURSE

Parameter and Return Types

JAX-RS 2.x | Input & Output Data Handling

Topics Covered: Simple Types · @Consumes/@Produces · @XXXParam Annotations
@DefaultValue · Return Types · Binary Content · File Delivery
Platform: Java 11 LTS | Tomcat 9.0 | JAX-RS 2.1
Namespace: javax.ws.rs.* (NOT jakarta.ws.rs.*)
Data format: XML (domain objects) | TEXT_PLAIN (diagnostics) | OCTET_STREAM (binary)
Duration: 1 Day (8 hours)
Level: Intermediate — Java Developer

Table of Contents

1. Simple Parameter Types

1.1 Learning Objectives

- Understand which Java types JAX-RS can automatically convert from HTTP parameter strings
- Use primitive types, boxed types, String, and enum parameters with @PathParam and @QueryParam
- Understand when automatic type conversion fails and what HTTP status code results
- Write custom ParamConverter implementations for types not supported natively

1.2 What is a Parameter Type?

Every piece of data that arrives in an HTTP request — a path segment, query string value, header value, or form field — is ultimately a String when it crosses the wire. JAX-RS relieves you of the burden of parsing these strings manually by automatically converting them to the declared Java type of each annotated parameter.

JAX-RS supports automatic conversion to any type that satisfies one of the following conditions:

- It is a primitive type: boolean, byte, char, short, int, long, float, double
- It is a boxed type: Boolean, Byte, Character, Short, Integer, Long, Float, Double
- It is String — no conversion needed
- It has a public static fromString(String) method
- It has a public static valueOf(String) method
- It has a public constructor that accepts a single String argument
- It is a List<T>, Set<T>, or SortedSet<T> of any of the above

1.3 Primitives and Boxed Types

```
// Primitive — if 'page' is absent, JAX-RS uses 0 (default primitive value)
// If 'page' is present but non-numeric, JAX-RS returns 400 Bad Request
@GET @Path("/products")
@Produces(MediaType.APPLICATION_XML)
public List<Product> list(
    @QueryParam("page") int     page,    // primitive
    @QueryParam("size") Integer size) {  // boxed — null if absent
    int s = size != null ? size : 20;
    return repo.find(page, s);
}

// Long path parameter
@GET @Path("{id}")
@Produces(MediaType.APPLICATION_XML)
public Product getById(@PathParam("id") long id) { ... }

// Boolean query flag
@GET @Produces(MediaType.APPLICATION_XML)
public List<Product> list(@QueryParam("active") boolean active) { ... }
// GET /products?active=true   → active = true
// GET /products?active=false  → active = false
// GET /products               → active = false (primitive default)
```

1.4 Enum Parameters

```
public enum OrderStatus { NEW, PROCESSING, SHIPPED, DELIVERED, CANCELLED }

// GET /orders?status=PROCESSING
@GET @Produces(MediaType.APPLICATION_XML)
public List<Order> byStatus(@QueryParam("status") OrderStatus status) {
    if (status == null) return repo.findAll();
    return repo.findByStatus(status);
}
// GET /orders?status=processing  → 400 Bad Request
// 'processing' does not match 'PROCESSING' — valueOf() throws
IllegalArgumentException
```

1.5 Types with fromString() or valueOf()

```
// UUID path parameter — UUID.fromString() is called by the runtime
@GET @Path("{id}")
@Produces(MediaType.APPLICATION_XML)
public Product getById(@PathParam("id") java.util.UUID id) { ... }

// Custom type with fromString()
public class ProductCode {
    private final String code;
    public ProductCode(String code) { this.code = code; }
    public static ProductCode fromString(String s) {
        if (!s.matches("[A-Z]{3}-\\d{4}"))
            throw new IllegalArgumentException("Invalid code: " + s);
        return new ProductCode(s);
    }
    public String getCode() { return code; }
}

// GET /products/ABC-1234
@GET @Path("{code}")
@Produces(MediaType.APPLICATION_XML)
public Product getByCode(@PathParam("code") ProductCode code) { ... }
```

1.6 Collection Parameters

```
// GET /products?tag=java&tag=oop&tag=clean-code
@GET @Produces(MediaType.APPLICATION_XML)
public List<Product> byTags(@QueryParam("tag") List<String> tags) {
    if (tags == null || tags.isEmpty()) return repo.findAll();
    return repo.findByTags(tags);
}

// GET /orders?status=NEW&status=PROCESSING
@GET @Produces(MediaType.APPLICATION_XML)
public List<Order> byStatuses(@QueryParam("status") List<OrderStatus>
statuses) {
    return repo.findByStatuses(statuses);
}
```

1.7 Custom ParamConverter

For types that don't satisfy any of the built-in conversion rules, implement a ParamConverter and register it via a ParamConverterProvider. Import from javax.ws.rs.ext (NOT jakarta.ws.rs.ext):

> **Package Namespace & Data Format**
> All code uses javax.ws.rs.* (NOT jakarta.ws.rs.*) — correct for Java 11 / Tomcat 9.
> Domain object endpoints use @Produces(APPLICATION_XML) / @Consumes(APPLICATION_XML).
> Diagnostic / echo endpoints use TEXT_PLAIN. Binary endpoints use APPLICATION_OCTET_STREAM.
> Import: import javax.ws.rs.*; import javax.ws.rs.core.*; import javax.ws.rs.ext.*;

```
// Custom converter for LocalDate (JAX-RS has no built-in LocalDate support)
import javax.ws.rs.ext.Provider;
import javax.ws.rs.ext.ParamConverter;
import javax.ws.rs.ext.ParamConverterProvider;

@Provider
public class LocalDateConverterProvider implements ParamConverterProvider {
    @Override
    @SuppressWarnings("unchecked")
    public <T> ParamConverter<T> getConverter(
            Class<T> rawType, java.lang.reflect.Type genericType,
            java.lang.annotation.Annotation[] annotations) {
        if (rawType != java.time.LocalDate.class) return null;
        return (ParamConverter<T>) new ParamConverter<java.time.LocalDate>()
{
            public java.time.LocalDate fromString(String s) {
                return s == null ? null : java.time.LocalDate.parse(s);
            }
            public String toString(java.time.LocalDate v) {
                return v == null ? null : v.toString();
            }
        };
    }
}

// After registering the provider, use LocalDate directly:
// GET /orders?from=2024-01-01&to=2024-12-31
@GET @Produces(MediaType.TEXT_PLAIN)
public Response byDateRange(
    @QueryParam("from") java.time.LocalDate from,
    @QueryParam("to")   java.time.LocalDate to) { ... }
```

1.8 Type Conversion Failures

Annotation	Conversion Failure Result
@PathParam	404 Not Found — the path template matched but the value cannot be bound
@QueryParam	400 Bad Request — the query string value is invalid
@HeaderParam	400 Bad Request — the header value is invalid
@FormParam	400 Bad Request — the form field value is invalid
@MatrixParam	400 Bad Request — the matrix parameter value is invalid

1.9 Review Questions

1. Which six categories of Java type can JAX-RS convert from a String parameter automatically?
2. A method has @QueryParam("count") int count. The request URL contains ?count=abc. What HTTP status code does the client receive?
3. You want to accept a comma-separated list of IDs as a single query parameter: ?ids=P001,P002,P003. The built-in List<String> support collects repeated parameters, not comma-separated values. How would you handle this?

2. @Consumes and @Produces Annotations

2.1 Learning Objectives

- Apply @Consumes and @Produces at class and method level
- Understand how they participate in content negotiation
- Handle multiple media types with quality factors
- Understand 415 Unsupported Media Type and 406 Not Acceptable responses

2.2 Purpose and Scope

Term	Definition
@Produces	Declares the media type(s) this method will write to the response body. Matched against the client's Accept header.
@Consumes	Declares the media type(s) this method can read from the request body. Matched against the request's Content-Type header.

Both annotations can appear at the class level (default for all methods) or at the method level (overrides the class-level declaration for that specific method). This module uses APPLICATION_XML as the primary format for domain objects:

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@Path("/products")
@Produces(MediaType.APPLICATION_XML)    // class default — all methods return
XML
@Consumes(MediaType.APPLICATION_XML)    // class default — all methods accept
XML
public class ProductResource {

    @GET                                // inherits @Produces(XML) from class
    public List<Product> list() { ... }

    @GET @Path("{id}")
    @Produces({MediaType.APPLICATION_XML,     // overrides class-level
@Produces
               MediaType.TEXT_PLAIN})         // also produces plain text
summary
    public Response get(@PathParam("id") String id) { ... }

    @POST
    @Consumes({MediaType.APPLICATION_XML,     // overrides class-level
@Consumes
               "application/vnd.example.product+xml"})  // and custom type
    public Response create(Product p, @Context UriInfo ui) { ... }
}
```

2.3 Content Negotiation — How the Runtime Selects

```
// Method producing XML and plain text
@GET @Path("{id}")
@Produces({MediaType.APPLICATION_XML, MediaType.TEXT_PLAIN})
public Response get(@PathParam("id") String id) { ... }

// Accept: application/xml   → returns XML
// Accept: text/plain        → returns plain text
// Accept: */*               → returns XML (first listed)
// Accept: text/html         → 406 Not Acceptable

// Q-values — client preference weighting
// Accept: application/xml;q=0.9, text/plain;q=1.0
// → returns plain text (higher q-value wins)

// Accept: application/xml;q=0.9, */*;q=0.1
// → returns XML (more specific match wins over wildcard)
```

2.4 Common Media Types

MediaType Constant	MIME String	Typical Use
APPLICATION_XML	application/xml	REST API domain objects (JAXB-annotated)
TEXT_XML	text/xml	Alternative XML MIME type (SOAP default)
TEXT_PLAIN	text/plain	Simple string responses, diagnostic output
TEXT_HTML	text/html	HTML page rendering
MULTIPART_FORM_DATA	multipart/form-data	File upload forms
APPLICATION_FORM_URLENCODED	application/x-www-form-urlencoded	HTML form POST bodies
APPLICATION_OCTET_STREAM	application/octet-stream	Binary file download
SERVER_SENT_EVENTS	text/event-stream	SSE streaming (JAX-RS 2.1+)
(custom)	application/vnd.company.v2+xml	API versioning via media type

2.5 Error Responses

415 Unsupported Media Type
Returned when the request body's Content-Type header does not match any media type listed in the method's @Consumes annotation.

Example: POST with Content-Type: text/plain on a method with @Consumes(APPLICATION_XML)

406 Not Acceptable
Returned when no method can produce a media type that satisfies the request's Accept header.
Example: Accept: text/html on a resource that only @Produces APPLICATION_XML

2.6 Wildcards and Vendor Types

```
// Wildcard — accept any media type in the request body
@POST
@Consumes("*/*")
public Response ingest(java.io.InputStream raw) {
    // caller decides what to send; we read it as raw bytes
}

// Vendor-specific XML type for API versioning
@GET @Path("{id}")
@Produces({
    "application/vnd.example.product-v2+xml",  // v2 client
    MediaType.APPLICATION_XML                    // generic client fallback
})
public Product get(@PathParam("id") String id) { ... }

// Structured text — CSV
@GET
@Produces("text/csv")
public Response exportCsv() {
    String csv = "id,name,price\n" + buildRows();
    return Response.ok(csv)
                   .header("Content-Disposition", "attachment;
filename=products.csv")
                   .build();
}
```

2.7 Review Questions

4. A class has @Produces(APPLICATION_XML) and a method has @Produces(TEXT_PLAIN). Which applies to that method?
5. A request arrives with Accept: text/html but the only method is annotated @Produces(APPLICATION_XML). What status code does the client receive?
6. Explain what q=0.0 means in an Accept header and how JAX-RS handles it.

3. @XXXParam Annotations

3.1 Learning Objectives

- Apply all seven @Param annotations to inject request data into method parameters
- Understand the source of data injected by each annotation
- Use @BeanParam to aggregate multiple annotations into a single parameter object
- Inject JAX-RS context objects using @Context

3.2 Overview of All Parameter Annotations

Term	Definition
@PathParam	Binds a URI template variable. Source: path segment matching {name} in @Path.
@QueryParam	Binds a URL query parameter. Source: ?name=value pairs after the ? in the URL.
@HeaderParam	Binds an HTTP request header. Source: any header line in the HTTP request.
@FormParam	Binds a form field. Source: application/x-www-form-urlencoded or multipart/form-data body.
@CookieParam	Binds a cookie value. Source: Cookie header sent by the browser or client.
@MatrixParam	Binds a matrix parameter. Source: ;name=value pairs embedded in a path segment.
@BeanParam	Aggregates multiple @Param annotations from a single bean class. Reduces method signature clutter.
@Context	Injects a JAX-RS context object: UriInfo, HttpHeaders, Request, SecurityContext, Providers.

3.3 @PathParam

```
// Single path variable
@GET @Path("/products/{id}") @Produces(MediaType.APPLICATION_XML)
public Product get(@PathParam("id") String id) { ... }

// Multiple path variables
@GET @Path("/orders/{year}/{month}") @Produces(MediaType.APPLICATION_XML)
public List<Order> byMonth(
    @PathParam("year")  int year,
    @PathParam("month") int month) { ... }

// Regex-constrained
@GET @Path("/products/{sku: [A-Z]+-\\d+}")
@Produces(MediaType.APPLICATION_XML)
public Product bySku(@PathParam("sku") String sku) { ... }

// Greedy — captures slashes
```

```
@GET @Path("/files/{path: .+}")
public Response getFile(@PathParam("path") String path) { ... }
```

3.4 @QueryParam

```
// Simple query parameter — null if absent
@GET @Produces(MediaType.APPLICATION_XML)
public List<Product> list(
    @QueryParam("category") String     category,
    @QueryParam("maxPrice") java.math.BigDecimal maxPrice) { ... }
// GET /products?category=books&maxPrice=49.99

// Repeating query parameter (multi-value)
@GET @Produces(MediaType.APPLICATION_XML)
public List<Order> byStatuses(
    @QueryParam("status") List<OrderStatus> statuses) { ... }
// GET /orders?status=NEW&status=PROCESSING

// With default value
@GET @Produces(MediaType.APPLICATION_XML)
public List<Product> paged(
    @QueryParam("page") @DefaultValue("0")  int page,
    @QueryParam("size") @DefaultValue("20") int size) { ... }
```

3.5 @HeaderParam

```
// Read a standard header
@GET @Path("{id}") @Produces(MediaType.APPLICATION_XML)
public Response get(
        @PathParam("id")            String id,
        @HeaderParam("Accept")      String accept,
        @HeaderParam("If-None-Match") String etag) {
    Product p = repo.findById(id);
    if (p.getEtag().equals(etag)) {
        return Response.notModified().build();  // 304
    }
    return Response.ok(p).header("ETag", p.getEtag()).build();
}

// Custom headers — API keys and correlation IDs
@POST @Consumes(MediaType.APPLICATION_XML)
@Produces(MediaType.APPLICATION_XML)
public Response create(
        Product product,
        @HeaderParam("X-API-Key")    String apiKey,
        @HeaderParam("X-Request-ID") String requestId,
        @Context UriInfo             uriInfo) {
    validateApiKey(apiKey);
    Product saved = repo.save(product);
    return Response.created(uriInfo.getAbsolutePathBuilder()
                                   .path(saved.getId()).build())
                   .entity(saved)
                   .header("X-Request-ID", requestId)  // echo back
                   .build();
}
```

3.6 @FormParam

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@POST @Path("/login")
@Consumes(MediaType.APPLICATION_FORM_URLENCODED)
@Produces(MediaType.TEXT_PLAIN)
public Response login(
        @FormParam("username") String username,
        @FormParam("password") String password) {
    if (authService.authenticate(username, password)) {
        return Response.ok("token=" + tokenService.issue(username)).build();
    }
    return Response.status(Response.Status.UNAUTHORIZED).build();
}
// curl -X POST .../login -d 'username=alice&password=secret'
```

3.7 @CookieParam

Import Cookie from javax.ws.rs.core (NOT jakarta.ws.rs.core):

```
import javax.ws.rs.core.Cookie;

// Read a session cookie
@GET @Path("/profile") @Produces(MediaType.APPLICATION_XML)
public Response profile(
        @CookieParam("SESSION_ID") String sessionId) {
    User user = sessionService.getUser(sessionId);
    if (user == null) return Response.status(401).build();
    return Response.ok(user).build();
}

// Inject as a Cookie object (includes path, domain, max-age)
@GET @Produces(MediaType.TEXT_PLAIN)
public Response withCookie(
        @CookieParam("PREF") Cookie pref) {
    String theme = pref != null ? pref.getValue() : "light";
    return Response.ok("theme=" + theme + "\n").build();
}
```

3.8 @MatrixParam

```
// GET /products;category=books;maxPrice=50
@GET @Path("/products") @Produces(MediaType.APPLICATION_XML)
public List<Product> filter(
        @MatrixParam("category") String category,
        @MatrixParam("maxPrice") java.math.BigDecimal maxPrice) { ... }

// Matrix params on specific path segments
// GET /orders/2024;quarter=Q1/summary
@GET @Path("/orders/{year}/summary") @Produces(MediaType.TEXT_PLAIN)
public Response summary(
        @PathParam("year")    int    year,
        @MatrixParam("quarter") String quarter) { ... }
```

3.9 @BeanParam — Aggregating Parameters

```
import javax.ws.rs.*;

// Parameter bean — all @Param annotations go here
public class ProductFilter {
    @QueryParam("category")
    public String category;

    @QueryParam("minPrice") @DefaultValue("0")
    public java.math.BigDecimal minPrice;

    @QueryParam("maxPrice")
    public java.math.BigDecimal maxPrice;

    @QueryParam("page") @DefaultValue("0")
    public int page;

    @QueryParam("size") @DefaultValue("20")
    public int size;

    @QueryParam("sort") @DefaultValue("name")
    public String sort;

    @HeaderParam("Accept-Language")
    public String language;
}

// Clean method signature — one parameter instead of seven
@GET @Produces(MediaType.APPLICATION_XML)
public List<Product> list(@BeanParam ProductFilter filter) {
    return repo.find(filter);
}
```

3.10 @Context — Injecting Framework Objects

Term	Definition
UriInfo	Full request URI, path parameters, query parameters, base URI. Essential for building Location headers.
HttpHeaders	All request headers as a MultivaluedMap. Also provides parsed Accept and Content-Type.
Request	HTTP method and conditional request evaluation (selectVariant, evaluatePreconditions).
SecurityContext	Principal, roles, authentication scheme. Used in auth-related logic.
Providers	Registry of MessageBodyReaders, MessageBodyWriters, ExceptionMappers.
ResourceContext	Create and initialise sub-resource instances with CDI injection support.
Application	The Application subclass instance. Access to properties set in Application.getProperties().

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@GET @Path("{id}") @Produces(MediaType.APPLICATION_XML)
public Response get(
        @PathParam("id")    String      id,
        @Context UriInfo    uriInfo,
        @Context HttpHeaders headers,
        @Context Request    request) {
    Product p = repo.findById(id);
    if (p == null) return Response.status(404).build();
    // Conditional GET using ETag
    EntityTag etag = new EntityTag(Integer.toHexString(p.hashCode()));
    Response.ResponseBuilder rb = request.evaluatePreconditions(etag);
    if (rb != null) return rb.build();  // 304 Not Modified
    return Response.ok(p).tag(etag).build();
}
```

3.11 Review Questions

7. List all seven @Param annotations and the source of data each injects.
8. When would you use @BeanParam instead of individual @QueryParam annotations?
9. A method needs the full request URI to build a redirect response. Which @Context injection gives you this, and what method do you call?

4. The @DefaultValue Annotation

4.1 Learning Objectives

- Apply @DefaultValue to query, header, form, cookie, and matrix parameters
- Understand the interaction between @DefaultValue and nullable vs primitive types
- Understand the string parsing rules for non-String default values
- Design robust, self-documenting API parameter defaults

4.2 Purpose

```
// Without @DefaultValue — page is 0, size is null, sort is null
@GET @Produces(MediaType.APPLICATION_XML)
public List<Product> list(
    @QueryParam("page") int     page,
    @QueryParam("size") Integer size,
    @QueryParam("sort") String  sort) {
    int s    = (size != null) ? size : 20;        // manual default
    String o = (sort != null) ? sort : "name";    // manual default
    return repo.find(page, s, o);
}

// With @DefaultValue — cleaner, self-documenting
@GET @Produces(MediaType.APPLICATION_XML)
public List<Product> list(
    @QueryParam("page") @DefaultValue("0")    int    page,
    @QueryParam("size") @DefaultValue("20")   int    size,
    @QueryParam("sort") @DefaultValue("name") String sort) {
    return repo.find(page, size, sort);
}
```

4.3 @DefaultValue with All Param Types

```
// @HeaderParam with default
@GET @Produces(MediaType.APPLICATION_XML)
public Response get(
    @PathParam("id")                                String id,
    @HeaderParam("Accept-Language")
    @DefaultValue("en-GB")                         String lang,
    @HeaderParam("X-Rate-Limit-Version")
    @DefaultValue("1")                             int    limitVersion) { ...
}

// @CookieParam with default
@GET @Produces(MediaType.TEXT_PLAIN)
public Response profile(
    @CookieParam("THEME") @DefaultValue("light") String theme) { ... }

// @FormParam with default
@POST @Consumes(MediaType.APPLICATION_FORM_URLENCODED)
@Produces(MediaType.TEXT_PLAIN)
public Response register(
    @FormParam("role") @DefaultValue("USER") String role,
    @FormParam("lang") @DefaultValue("en")   String lang) { ... }
```

4.4 String Parsing of Default Values

```
// Works — "0" is a valid int string
@QueryParam("page") @DefaultValue("0") int page

// Works — "true" is a valid boolean string
@QueryParam("active") @DefaultValue("true") boolean active

// Works — "NEW" is a valid OrderStatus.valueOf() argument
@QueryParam("status") @DefaultValue("NEW") OrderStatus status

// WRONG — "zero" cannot be parsed as int
// @QueryParam("page") @DefaultValue("zero") int page  ← runtime error
```

4.5 @DefaultValue and Null

Important: @DefaultValue does NOT affect injection for PRESENT parameters
@DefaultValue only fires when the parameter is ABSENT from the request.
If the client sends ?page= (empty string), the empty string is injected, not the default.
If the client sends ?page=5, 5 is injected.
Only if the parameter is completely missing from the URL is the default used.

```
// Test matrix for @QueryParam("size") @DefaultValue("20") Integer size:
// GET /products          → size = 20      (default applied)
// GET /products?size=50  → size = 50      (explicit value)
// GET /products?size=    → size = null    (empty string → null for Integer)
// GET /products?size=abc → 400 Bad Request (cannot parse)
```

4.6 Best Practices

- Always declare @DefaultValue for optional numeric parameters to avoid primitive default confusion
- Prefer Integer (boxed) over int when you need to distinguish 'absent' from 'explicitly zero'
- Document your default values in API documentation (OpenAPI) — @DefaultValue makes this easy
- Use sensible page sizes as defaults: @DefaultValue("20") is a common convention
- Never use @DefaultValue to inject secrets — the string appears in source code and logs

4.7 Review Questions

10. A method has @QueryParam("page") @DefaultValue("0") int page. The client sends GET /products?page=. What value does page receive?
11. Why is Integer preferred over int when a query parameter should be treated differently when absent vs explicitly zero?
12. You want a boolean filter parameter that defaults to true. Write the complete annotation pair.

5. Return Types

5.1 Learning Objectives

- Use void, POJO, Collection, and Response return types appropriately
- Build Response objects with custom status codes, headers, and cookies
- Understand GenericEntity for preserving generic type information
- Return CompletionStage<T> for asynchronous processing

5.2 Return Type Options

Return Type	Status Code	Body	Header Control
void	204 No Content	None	None (no Response object)
POJO (e.g., Product)	200 OK	Serialised by MessageBodyWriter	None (no Response object)
Collection<T>	200 OK	Serialised collection	None
Response	As set by builder	As set by entity()	Full control via header()
Response (null entity)	200 OK + empty body	None	Full control
CompletionStage<T>	Async — determined later	Async entity	Via AsyncResponse

5.3 Returning a POJO

When a method returns a non-void, non-Response type, JAX-RS serialises it with the first matching MessageBodyWriter. For @Produces(APPLICATION_XML) the built-in JAXB provider handles POJO → XML automatically when the class carries @XmlRootElement:

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

// Returns 200 OK with XML body (JAXB marshals Product to XML)
@GET @Path("{id}")
@Produces(MediaType.APPLICATION_XML)
public Product get(@PathParam("id") String id) {
    Product p = repo.findById(id);
    if (p == null) throw new NotFoundException();  // → 404
    return p;  // → 200 OK with <product id="P001">...</product>
}

// Returns 200 OK with XML list
@GET @Produces(MediaType.APPLICATION_XML)
public List<Product> list() {
    return repo.findAll();  // JAXB provider marshals List<Product> to XML
}
```

5.4 Building a Response Object

Use Response when you need fine-grained control over the status code, headers, cookies, or body. Import from javax.ws.rs.core (NOT jakarta.ws.rs.core):

```
import javax.ws.rs.core.Response;
import javax.ws.rs.core.Response.Status;
import javax.ws.rs.core.NewCookie;

// 201 Created with Location header
@POST @Consumes(MediaType.APPLICATION_XML)
@Produces(MediaType.APPLICATION_XML)
public Response create(Product p, @Context UriInfo ui) {
    Product saved = repo.save(p);
    java.net.URI loc =
ui.getAbsolutePathBuilder().path(saved.getId()).build();
    return Response.created(loc)          // status 201 + Location header
                   .entity(saved)         // response body (JAXB → XML)
                   .build();
}

// 204 No Content
@DELETE @Path("{id}")
public Response delete(@PathParam("id") String id) {
    if (!repo.delete(id)) return Response.status(404).build();
    return Response.noContent().build();  // 204 — no body
}

// Custom status + headers
@PUT @Path("{id}")
@Consumes(MediaType.APPLICATION_XML) @Produces(MediaType.APPLICATION_XML)
public Response update(@PathParam("id") String id, Product p) {
    Product updated = repo.update(id, p);
    if (updated == null) return Response.status(Status.NOT_FOUND).build();
    return Response.ok(updated)            // 200 with XML body
                   .header("X-Updated-At",
java.time.Instant.now().toString())
                   .header("Cache-Control", "no-cache")
                   .build();
}

// Setting a cookie
@POST @Path("/login")
@Produces(MediaType.TEXT_PLAIN)
public Response login(@FormParam("username") String u,
                      @FormParam("password") String pw) {
    String token = authService.login(u, pw);
    NewCookie session = new NewCookie("SESSION", token,
        "/", null, 1, null, 3600, null, false, true);
    return Response.ok("status=ok\n")
                   .cookie(session)
                   .build();
}
```

5.5 Response.ResponseBuilder — Key Methods

Term	Definition
Response.ok(entity)	200 OK with entity body
Response.created(uri)	201 Created with Location header set to uri

Response.noContent()	204 No Content (no body)
Response.notModified()	304 Not Modified
Response.status(int/Status)	Any status code, no body by default
.entity(Object)	Set the response body (serialised by MessageBodyWriter)
.type(MediaType)	Explicitly set the Content-Type response header
.header(name, value)	Add or set any response header
.cookie(NewCookie)	Add a Set-Cookie response header
.tag(EntityTag)	Set the ETag response header
.lastModified(Date)	Set the Last-Modified response header
.cacheControl(CacheControl)	Set Cache-Control directives
.link(uri, rel)	Add a Link header (HATEOAS)
.build()	Finalise and return the immutable Response

5.6 GenericEntity — Preserving Generic Type

The built-in JAXB provider needs the generic element type of a returned List<T> to marshal it correctly. GenericEntity preserves this type information at runtime:

```
// Usually fine — JAXB infers List<Product> from the method return type
@GET @Produces(MediaType.APPLICATION_XML)
public Response list() {
    List<Product> products = repo.findAll();
    return Response.ok(products).build();
    // Works for most providers — type inferred from return type annotation
}

// Explicit type preservation — safe in all cases
@GET @Produces(MediaType.APPLICATION_XML)
public Response listSafe() {
    List<Product> products = repo.findAll();
    GenericEntity<List<Product>> entity =
        new GenericEntity<List<Product>>(products) {};  // {} preserves
generics
    return Response.ok(entity).build();
}
```

5.7 Asynchronous Return Types (JAX-RS 2.0+)

```
// CompletionStage<T> — non-blocking response (JAX-RS 2.1+)
@GET @Path("/expensive/{id}")
@Produces(MediaType.APPLICATION_XML)
public java.util.concurrent.CompletionStage<Response> getAsync(
        @PathParam("id") String id) {
    return java.util.concurrent.CompletableFuture
        .supplyAsync(() -> repo.findById(id))
        .thenApply(p -> {
            if (p == null) return Response.status(404).build();
            return Response.ok(p).build();
        });
```

```
}

// AsyncResponse — explicit suspend/resume (JAX-RS 2.0)
@GET @Path("/stream/{id}")
@Produces(MediaType.APPLICATION_XML)
public void getWithAsyncResponse(
        @PathParam("id") String id,
        @Suspended AsyncResponse ar) {
    executor.submit(() -> {
        Product p = slowRepo.findById(id);
        if (p == null) ar.resume(Response.status(404).build());
        else           ar.resume(p);
    });
}
```

5.8 Review Questions

13. A resource method returns null. What does JAX-RS do?
14. When would you use GenericEntity rather than returning a List<T> directly?
15. Write a Response that returns 302 Found with a Location header pointing to /products/P001.

6. Binary Content

6.1 Learning Objectives

- Read binary data from a request body as InputStream or byte[]
- Write binary data to a response body using InputStream, byte[], or StreamingOutput
- Use StreamingOutput for large binary responses that should not be buffered in memory
- Set correct Content-Type and Content-Disposition headers for binary responses

6.2 Reading Binary Input

```
// Receive binary as byte[] — entire body loaded into memory
@POST @Path("/images/{productId}")
@Consumes(MediaType.APPLICATION_OCTET_STREAM)
public Response uploadBytes(
        @PathParam("productId") String id,
        byte[] imageBytes) {
    imageService.store(id, imageBytes);
    return Response.noContent().build();
}

// Receive binary as InputStream — streaming, memory-efficient
@POST @Path("/images/{productId}")
@Consumes(MediaType.APPLICATION_OCTET_STREAM)
public Response uploadStream(
        @PathParam("productId") String id,
        java.io.InputStream imageStream) {
    imageService.store(id, imageStream);  // write to disk without buffering
    return Response.noContent().build();
}

// Receive multipart form data (file upload from HTML form)
@POST @Path("/upload")
@Consumes(MediaType.MULTIPART_FORM_DATA)
public Response uploadForm(
        @FormParam("file")     java.io.InputStream fileStream,
        @FormParam("filename") String filename) {
    storage.save(filename, fileStream);
    return Response.ok("saved=" + filename + "\n").build();
}
```

6.3 Writing Binary Output — byte[] and InputStream

```
// Return binary as byte[] — entire content buffered in memory first
@GET @Path("/images/{id}")
@Produces("image/jpeg")
public byte[] getImageBytes(@PathParam("id") String id) {
    return imageService.loadBytes(id);  // fine for small images
}

// Return binary as InputStream — JAX-RS streams it to the client
@GET @Path("/images/{id}")
@Produces("image/*")
public Response getImageStream(@PathParam("id") String id) {
    java.io.InputStream stream = imageService.openStream(id);
    if (stream == null) return Response.status(404).build();
```

```
    String type = imageService.detectMimeType(id);  // e.g. "image/png"
    return Response.ok(stream, type)
                   .header("Content-Length", imageService.sizeOf(id))
                   .build();
}
```

6.4 StreamingOutput — Lazy Writing

StreamingOutput is a callback interface. Import from javax.ws.rs.core (NOT jakarta.ws.rs.core):

```
import javax.ws.rs.core.StreamingOutput;

@GET @Path("/reports/{id}/pdf")
@Produces("application/pdf")
public Response downloadReport(@PathParam("id") String id) {
    Report report = reportService.find(id);
    if (report == null) return Response.status(404).build();
    StreamingOutput stream = new StreamingOutput() {
        @Override
        public void write(java.io.OutputStream out)
                throws java.io.IOException, WebApplicationException {
            pdfGenerator.generate(report, out);
            out.flush();
        }
    };
    return Response
        .ok(stream)
        .header("Content-Disposition",
                "attachment; filename=\"report-" + id + ".pdf\"")
        .build();
}

// Lambda equivalent (Java 11)
StreamingOutput stream = out -> {
    pdfGenerator.generate(report, out);
    out.flush();
};
```

6.5 Content-Disposition Header

Term	Definition
inline	Suggests the browser render the content in-page (e.g., display a PDF in the browser tab).
attachment	Instructs the browser to download and save the file. The filename parameter sets the save-as name.

```
// Force download with a specific filename
.header("Content-Disposition", "attachment; filename=\"products-
export.csv\"")

// Display inline (e.g., show PDF in browser)
.header("Content-Disposition", "inline; filename=\"invoice-2024-001.pdf\"")

// RFC 5987 encoding for non-ASCII filenames
.header("Content-Disposition",
```

```
        "attachment; filename*=UTF-8'" +
        java.net.URLEncoder.encode(filename, "UTF-8").replace("+", "%20"))
```

6.6 Review Questions

16. Explain the difference between returning an InputStream and returning a StreamingOutput. When would you choose one over the other?
17. What Content-Type header should a JPEG image response carry?
18. Write a resource method that streams a video file from disk, sets the correct Content-Type, and sets Content-Disposition to trigger a download.

7. Delivering a File

7.1 Learning Objectives

- Use java.io.File as a return type or entity in a Response
- Use java.nio.file.Path for modern file delivery
- Set all required headers for correct browser behaviour (Content-Type, Content-Length, Content-Disposition)
- Implement range request support for resumable downloads
- Apply security best practices to prevent path traversal attacks

7.2 Returning java.io.File

```
// Simplest file delivery — returns File directly
@GET @Path("/documents/{name}")
@Produces(MediaType.APPLICATION_OCTET_STREAM)
public java.io.File getDocument(@PathParam("name") String name) {
    java.io.File f = documentService.resolve(name);
    if (!f.exists()) throw new NotFoundException();
    return f;  // JAX-RS streams the file, sets Content-Length automatically
}

// Better — Response wrapper for full header control
@GET @Path("/documents/{name}")
public Response getDocumentWithHeaders(@PathParam("name") String name)
        throws java.io.IOException {
    java.io.File f = documentService.resolve(name);
    if (!f.exists()) return Response.status(404).build();
    String mimeType = java.nio.file.Files.probeContentType(f.toPath());
    if (mimeType == null) mimeType = MediaType.APPLICATION_OCTET_STREAM;
    return Response
        .ok(f, mimeType)
        .header("Content-Length",   f.length())
        .header("Content-Disposition",
                "attachment; filename=\"" + f.getName() + "\"")
        .header("Cache-Control", "private, max-age=3600")
        .build();
}
```

7.3 Using java.nio.file.Path (Java 11)

```
import java.nio.file.*;
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@GET @Path("/exports/{filename}")
public Response downloadExport(@PathParam("filename") String filename)
        throws java.io.IOException {
    Path file = exportDir.resolve(filename);
    // ── Security: prevent path traversal ──────────────────────────
    if (!file.normalize().startsWith(exportDir.normalize())) {
        return Response.status(Response.Status.FORBIDDEN).build();
    }
    if (!Files.exists(file)) return Response.status(404).build();
    String mimeType = Files.probeContentType(file);
```

```
    if (mimeType == null) mimeType = MediaType.APPLICATION_OCTET_STREAM;
    return Response
        .ok(file.toFile(), mimeType)
        .header("Content-Length",   Files.size(file))
        .header("Last-Modified",
Files.getLastModifiedTime(file).toString())
        .header("Content-Disposition",
                "attachment; filename=\"" + file.getFileName() + "\"")
        .build();
}
```

7.4 Streaming Large Files with StreamingOutput

```
import javax.ws.rs.core.StreamingOutput;

@GET @Path("/bulk-export")
public Response bulkExport(@QueryParam("format") @DefaultValue("csv") String
fmt) {
    StreamingOutput stream = out -> {
        try (java.io.BufferedWriter writer = new java.io.BufferedWriter(
                new java.io.OutputStreamWriter(out,
java.nio.charset.StandardCharsets.UTF_8))) {
            writer.write("id,name,price,category\n");
            repo.streamAll().forEach(p -> {
                try {
                    writer.write(String.format("%s,%s,%.2f,%s\n",
                        p.getId(), p.getName(), p.getPrice(),
p.getCategory()));
                } catch (java.io.IOException e) {
                    throw new WebApplicationException(e);
                }
            });
        }
    };
    return Response
        .ok(stream, "text/csv")
        .header("Content-Disposition", "attachment; filename=products.csv")
        .build();
}
```

7.5 Security — Path Traversal Prevention

Path Traversal Attack Example

GET /documents/../../etc/passwd
Without protection, this resolves to the system password file.

Prevention — always normalise and check against the base directory:

```
Path requested = baseDir.resolve(filename);
if (!requested.normalize().startsWith(baseDir.normalize())) {
    return Response.status(403).build();  // Forbidden
}
```

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import java.nio.file.*;
```

```
// Safe file resolution pattern — always use when filename comes from user
input
private static final Path BASE_DIR = Path.of("/opt/app/exports");

private Path safeResolve(String filename) {
    // Reject filenames containing path separators
    if (filename.contains("/") || filename.contains("\\")) {
        throw new BadRequestException("Filename must not contain path
separators");
    }
    Path resolved = BASE_DIR.resolve(filename).normalize();
    if (!resolved.startsWith(BASE_DIR.normalize())) {
        throw new ForbiddenException("Access denied");
    }
    return resolved;
}

@GET @Path("/files/{filename}")
public Response get(@PathParam("filename") String filename) throws
java.io.IOException {
    Path file = safeResolve(filename);
    if (!Files.exists(file)) return Response.status(404).build();
    return Response.ok(file.toFile(), Files.probeContentType(file))
                   .header("Content-Length", Files.size(file))
                   .build();
}
```

7.6 MIME Type Detection

```
// Java 11 built-in MIME detection (reads file magic bytes)
String mimeType = Files.probeContentType(file);
// Returns null if type cannot be determined — fall back to octet-stream
if (mimeType == null) mimeType = MediaType.APPLICATION_OCTET_STREAM;

// Manual extension mapping (fallback)
private static final java.util.Map<String,String> MIME_MAP =
java.util.Map.of(
    "pdf",  "application/pdf",
    "png",  "image/png",
    "jpg",  "image/jpeg",
    "csv",  "text/csv",
    "xlsx", "application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet"
);
private String mimeForExtension(String filename) {
    int dot = filename.lastIndexOf('.');
    if (dot < 0) return MediaType.APPLICATION_OCTET_STREAM;
    return MIME_MAP.getOrDefault(filename.substring(dot+1).toLowerCase(),
                                 MediaType.APPLICATION_OCTET_STREAM);
}
```

7.7 Complete File Download Resource

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import java.nio.file.*;
```

```
@Path("/downloads")
public class FileDownloadResource {
    private static final Path DOWNLOAD_DIR =
        Path.of(System.getProperty("download.dir", "/opt/app/downloads"));

    @GET @Path("{filename}")
    public Response download(
            @PathParam("filename") String filename,
            @QueryParam("inline") @DefaultValue("false") boolean inline)
            throws java.io.IOException {
        Path file = safeResolve(filename);
        if (!Files.exists(file))
            return Response.status(404).build();
        String mimeType = Files.probeContentType(file);
        if (mimeType == null) mimeType = MediaType.APPLICATION_OCTET_STREAM;
        String disposition = inline ? "inline" : "attachment";
        disposition += "; filename=\"" + file.getFileName() + "\"";
        return Response
            .ok(file.toFile(), mimeType)
            .header("Content-Length",      Files.size(file))
            .header("Content-Disposition", disposition)
            .header("Cache-Control",       "private, max-age=3600")
            .header("Last-Modified",
                    Files.getLastModifiedTime(file).toString())
            .build();
    }
}
```

7.8 Review Questions

19. What is path traversal and how do you prevent it when resolving user-supplied filenames?
20. A user requests GET /files/report.pdf but the file is 800 MB. Should you return File, InputStream, or StreamingOutput? Explain your choice.
21. Write the Content-Disposition header value to trigger a download saved as 'export-2024.csv'.
22. What does Files.probeContentType() return when the MIME type cannot be determined, and what should your fallback be?